



西北工业大学

第十届“龙芯杯”全国大学生 计算机系统能力大赛

作品报告

西北工业大学一队

团队成员： 杜仕铭、祝建祥、赵森、刘奕凡

指导教师： 王党辉、王继禾

2026年8月

目录

1 概述.....	1
1.1 设计目标.....	1
1.2 当前实测概述.....	1
1.3 主要设计特征.....	1
2 处理器总体架构.....	2
2.1 指令主通路.....	2
2.2 关键结构参数.....	2
2.3 正确性边界.....	3
3 取指前端与分支预测.....	3
3.1 八字取指与顺序整理.....	3
3.2 分支预测结构.....	4
3.3 前端阻塞与恢复.....	4
4 四路译码、重命名与分派.....	5
4.1 四路译码.....	5
4.2 物理寄存器重命名.....	5
4.3 分支检查点.....	5
4.4 原子分派.....	6
5 动态调度与执行通路.....	6
5.1 分布式发射队列.....	6
5.2 执行资源与完成总线.....	7
5.3 BRU 预期架构.....	7
6 ROB、四路提交与精确恢复.....	7
6.1 六十四项 ROB.....	7
6.2 四路提交.....	8
6.3 两级恢复控制.....	8
7 访存流水线与内嵌 LQ/SQ.....	9
7.1 LSU 流水组织.....	9
7.2 十六项 SQ.....	10
7.3 十六项 LQ 与逐字节前递.....	10
7.4 特殊访存操作.....	11
8 Cache 层次与外部存储访问.....	11
8.1 一级 Cache.....	11
8.2 Victim Buffer 与预取.....	11
8.3 二级 Cache 与 AXI.....	12
9 MMU、TLB、CSR 与异常中断.....	12
9.1 地址转换顺序.....	12
9.2 十六项全相联 TLB.....	12
9.3 异常判定.....	13
9.4 CSR、ERTN 与 refetch.....	13
10 总结.....	13
参考文献.....	14

1 概述

1.1 设计目标

本设计面向第十届“龙芯杯”全国大学生计算机系统能力大赛，目标是在 32 位 LoongArch 指令系统基础上构建一颗具备乱序执行、精确异常和完整存储层次的处理器核。与只追求流水级数或局部并行度的实现不同，本设计把“宽前端、动态调度、按序提交、快速恢复”作为统一的微架构约束：前端以 8 条指令为一组取指，译码、重命名、分派和提交宽度均为 4，后端使用分布式发射队列和 7 路普通结果/完成通路，最终由 64 项重排序缓冲区维护体系结构可见顺序。

处理器的正确性目标包括：支持功能测试要求的指令与异常行为；在分支预测错误、精确异常、ERTN 和 refetch 等事件下恢复一致状态；保证投机访存不破坏内存顺序；通过两级 Cache、TLB 和 AXI 接口完成片上系统访问。性能目标则集中在扩大有效指令窗口、降低前端气泡、缩短相关指令唤醒链路以及提高存储访问命中率。

1.2 当前实测概述

本报告暂以指定成绩工作簿中的数据作为当前实测基线。功能测试共 58 个测试点，完成 58/58，通过率为 100%；系统测试能够启动 Linux 并执行 ls 命令。20 项性能程序均获得有效计数，本文只给出汇总指标，不逐项展开程序名称和单项成绩。由工作簿原始计数重新计算得到系统计数比 3.437890、IPC 比 1.405570；CPU 运行频率为 80MHz，CPU/SoC 时钟比几何平均值为 0.798663，与 80MHz/100MHz 的配置关系一致。

表 1-1 当前实测指标汇总

指标	结果
功能测试	58/58，100%
系统测试	可启动 Linux 并执行 ls
性能程序覆盖	20 项均有有效计数
系统计数比	3.437890
IPC 比	1.405570
CPU 频率	80MHz
CPU/SoC 时钟比	0.798663

1.3 主要设计特征

本设计的主要特征可以概括为四点。第一，前端以 8 路预测银行配合 8 字取指返回，在命中条件允许时每拍向 128 项指令 FIFO 写入最多 8 条指令，并稳定向后端提供最多 4 条有序指令。第二，重命名阶段使用 128 项物理寄存器和 4 个并行分配区域，通过同拍 RAW/WAW 旁路处理 4 条指令之间的依赖，并为最多 32 个在途分支保存 RMT 检查点。第三，后端将整数、访存、乘除/分支、快速执行和特权操作分配到不同发射队列，完成结果统一广播到 ROB、PRF 和所有等待队列。第

四, LSU 内部集成 16 项 LQ 与 16 项 SQ, 在访存流水线内完成存储数据补齐、逐字节前递、冲突等待与槽位复用保护。

2 处理器总体架构

2.1 指令主通路

处理器主通路由前端、顺序建档区、乱序执行区和按序提交区组成。前端按预测 PC 访问指令 Cache, 每次返回 256bit, 即 8 个 32bit 指令字; 指令进入 128 项 FIFO 后, 以低编号连续前缀方式向 4 路译码阶段输出。译码结果经过四路重命名, 获得源物理寄存器、目的物理寄存器和旧目的映射, 再由分派阶段同时申请 ROB 与对应发射队列资源。只有所有必要资源均可用时, 一条指令才进入乱序窗口, 因而不会出现 ROB 已分配而发射队列未登记的半完成状态。

进入发射队列后, 指令根据源操作数就绪情况竞争对应执行资源。执行可乱序完成, 结果通过 7 路普通完成/写回通路写入物理寄存器文件、标记 ROB 完成, 并向各发射队列广播目的物理寄存器标签。ROB 始终保存程序顺序, 最多从队首提交 4 条低编号连续指令; 提交时更新精确 RMT、释放旧物理寄存器, 并把 Store 从投机状态推进到可对存储系统生效的已提交状态。

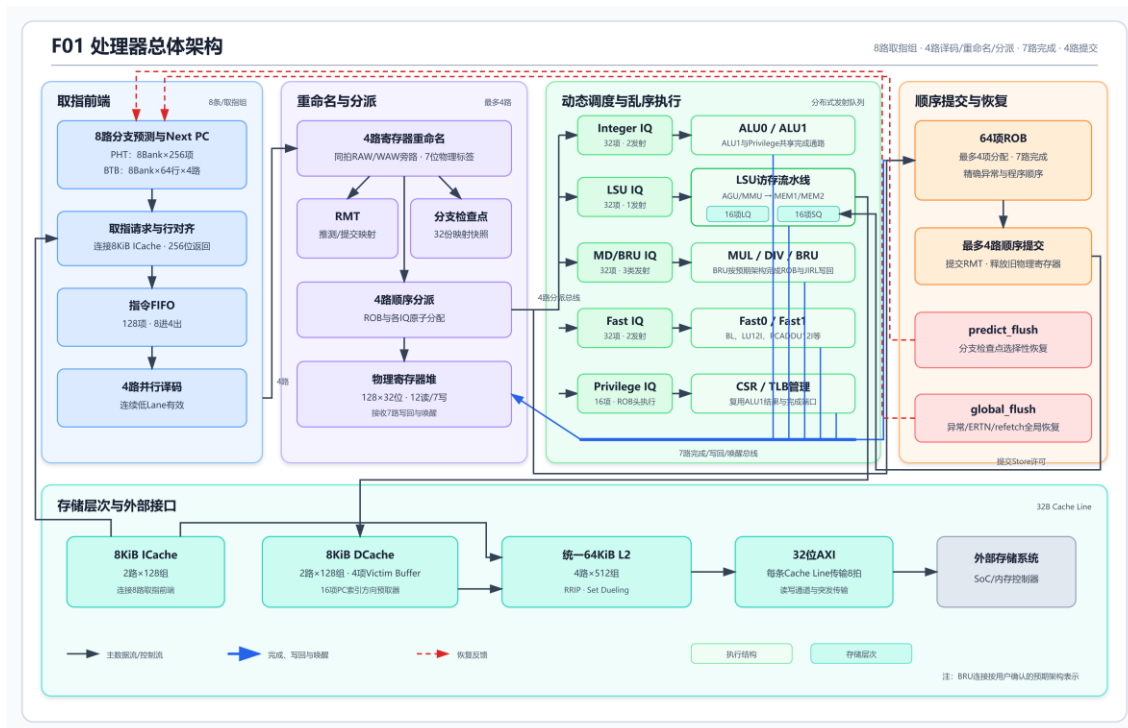


图 2-1 处理器总体微架构与主要数据流

2.2 关键结构参数

结构参数不是孤立的容量选择, 而是围绕 4 路后端宽度和 7 路完成带宽配套确定。128 项 FIFO 吸收前端 8 入与后端 4 出的速率差; 64 项 ROB 提供足够的乱序窗口; 128 项物理寄存器既容纳 32 个体系结构寄存器的当前映射, 也为在途目的寄存器留出空间。分布式发射队列按执行资源拆分, 避免单一大队列的比较器和选择逻辑成为时序热点。

表 2-1 处理器关键结构参数

模块	配置	作用
取指与 FIFO	8 条取指, 128 项 FIFO, 最多 4 条输出	解耦前端与译码端
译码/重命名/分派	4 路	建立并投递最多 4 条指令
物理寄存器文件	128×32bit, 12 读 7 写	支撑 4 路源读取与 7 路结果写回
分支检查点	32 份, 每份 32×7bit 映射	预测错误时选择性恢复 RMT
发射队列	Integer/LSU/MD-BRU/Fast 各 32 项, Privilege16 项	分布式动态调度
ROB	64 项, 最多 4 路提交	维护程序顺序与精确状态
LSQ	LSU 内部 16 项 LQ+16 项 SQ	访存相关、前递与顺序约束
一级 Cache	I/D 各 2 路×128 组×32B, 8KiB	低延迟取指与数据访问
二级 Cache	4 路×512 组×32B, 64KiB	降低外部存储访问频率
TLB	16 项全相联, 支持 4KiB/4MiB 页	虚实地址转换与权限检查

2.3 正确性边界

乱序执行只改变内部完成先后, 不改变体系结构可见顺序。寄存器结果必须经过 ROB 队首提交后才成为精确状态的一部分; Store 即使地址和数据提前准备完成, 也只有提交后才允许从 SQ 队首排出。分支预测错误只清除错误分支之后的年轻状态, 而精确异常、ERTN 与 refetch 清除全部年轻状态并选择新的取指地址。该边界把高并行度和精确状态分离开, 使后端可以积极执行, 同时由 ROB、提交 RMT 和 LSQ 共同约束对外可见行为。

3 取指前端与分支预测

3.1 八字取指与顺序整理

指令 Cache 一次返回 256bit 数据, 天然对应 8 个指令字。前端根据当前 PC 的字内偏移生成有效掩码, 将预测路径上的有效指令按程序顺序写入 8 银行、每银行 16 项的 128 项 FIFO。FIFO 同时处理最多 8 条写入和最多 4 条读出, 并通过计数、读写指针与低编号连续有效规则保持输出稳定。这样, Cache 返回宽度、预测粒度和后端译码宽度可以解耦, 短时的 Cache 等待也不会立即传递成后端气泡。

3.2 分支预测结构

条件分支方向预测由 8 个 PHT 银行完成, 每个银行包含 256 个 2bit 饱和计数器, 总计 2048 项。目标预测由 8 个 BTB 银行完成, 每银行 64 组、每组 4 路, 总计 2048 项, 并使用树形 PLRU 维护替换次序。银行化结构让同一个取指组中的多个位置可以并行查询; 方向与目标信息组合后, 前端选择取指组内最早的预测跳转作为下一拍 PC 来源, 避免越过更早的控制流指令。

预测更新采用提交或已确认分支信息，避免错误路径污染训练状态。直接跳转 B/BL 在译码阶段即可形成重定向，其中 B 不占用普通发射队列，BL 除重定向外还通过快速执行路径写入链接值。条件分支和 JIRL 由 BRU 判定实际方向与目标，若与预测快照不一致则产生 `predict_flush`，并使用相应检查点恢复重命名状态。

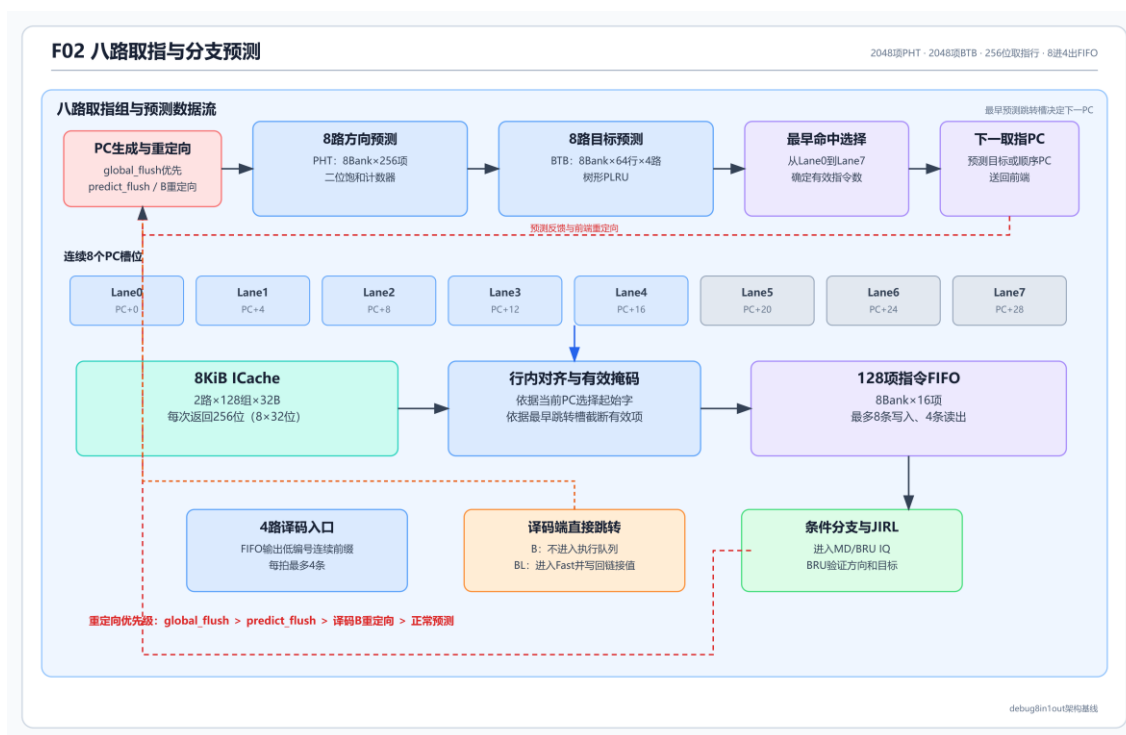


图 3-1 取指前端、分支预测与重定向控制

3.3 前端阻塞与恢复

前端需要同时响应容量压力和控制流变化。当 FIFO 剩余空间不足以接收新的取指组，或指令 Cache/TLB 尚未给出有效结果时，取指状态保持；后端阻塞时，FIFO 继续保存尚未消费的有序指令。发生重定向时，优先级依次为 `global_flush`、`predict_flush`、B 类译码重定向和普通预测路径。高优先级恢复事件会清理低优先级路径上已经形成但尚未提交的前端状态，确保新 PC 和恢复后的后端映射属于同一条体系结构路径。

4 四路译码、重命名与分派

4.1 四路译码

译码阶段每拍接收 FIFO 给出的最多 4 条连续指令，生成源/目的体系结构寄存器、立即数、执行类型、访存属性、分支属性以及异常信息。输出仍保持程序顺序；若某一 Lane 无效或资源不足，后续更高编号 Lane 不能绕过它独立进入重命名和分派。该连续前缀约束简化了 ROB 编号分配、分支检查点编号和同拍依赖处理。

4.2 物理寄存器重命名

寄存器重命名使用 $32 \times 7\text{bit}$ 推测 RMT 记录当前投机映射，物理寄存器 0 保持常量语义，其余物理寄存器由空闲表分配。4 个重命名 Lane 分别对应 4 个物理寄存器分配区域，使多个目的寄存器可以在同一拍并行获得新标签。对每条写寄存器指令，重命名阶段同时保存新 pdest 与 old pdest：新标签用于后续数据相关和写回，旧标签在该指令提交后释放。

同一拍的 4 条指令可能存在 RAW 或 WAW 依赖。Lane1 接收 Lane0 的映射旁路，Lane2 接收 Lane0~1，Lane3 接收 Lane0~2，因此年轻指令读取到的是同拍更老指令已经更新后的最新映射。该逐级旁路既保持程序顺序，也避免通过串行更新单一 RMT 而限制 4 路吞吐。

4.3 分支检查点

每个需要恢复的控制流指令在成功重命名后建立一份检查点，最多保存 32 份 $32 \times 7\text{bit}$ 的 RMT 映射。预测正确时，相应检查点随分支提交或确认而释放；预测错误时，predict_flush 用分支检查点恢复 RMT，并清除 ROB、发射队列和 LSU 中比该分支年轻的状态。global_flush 则从提交 RMT 恢复，适用于异常、ERTN 和 refetch 等要求回到精确提交边界的事件。

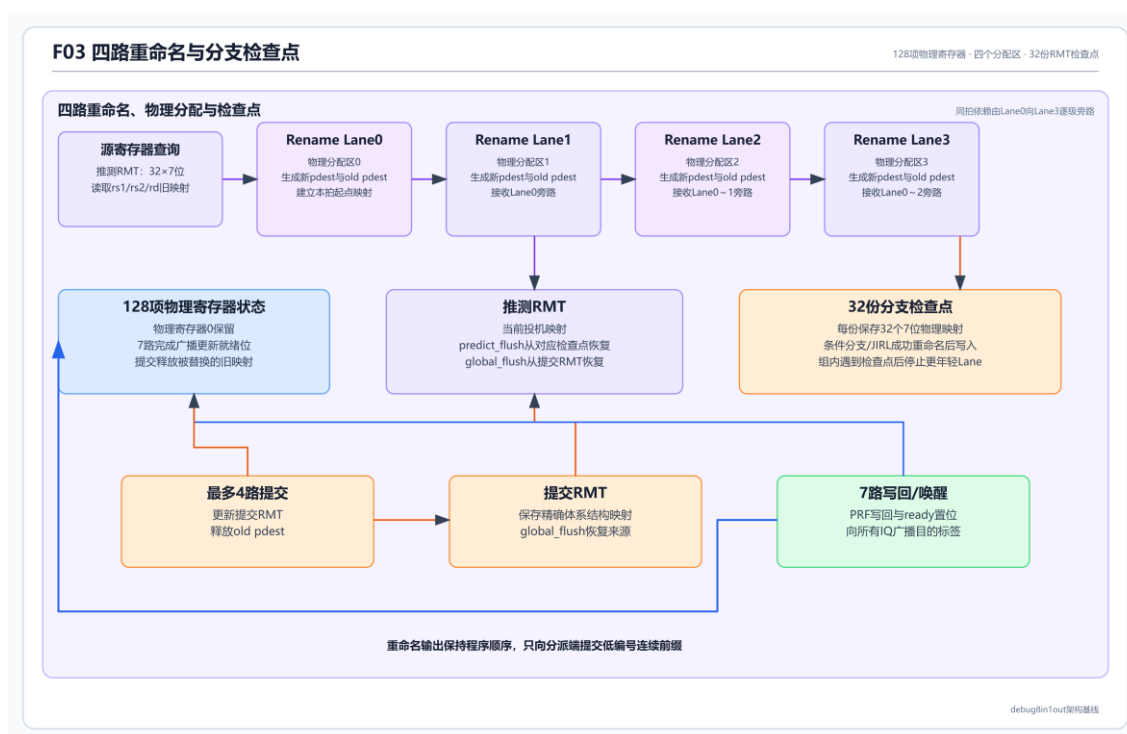


图 4-1 四路重命名、物理分配与分支检查点

4.4 原子分派

分派阶段根据指令类型选择 Integer、LSU、MD/BRU、Fast 或 Privilege 发射队列，并同步申请 ROB 项、物理寄存器和必要的分支/访存资源。资源判断以整个连续前缀为单位：若某条指令不能同时获得所有必要资源，则该条及其后的年轻指令本拍均不分派。该策略防止不同后端结构对同一条指令形成不一致的接收结果，是流水线恢复和计数管理保持简单的关键。

5 动态调度与执行通路

5.1 分布式发射队列

后端按执行资源设置五类发射队列。Integer IQ 为 32 项，支持 4 路分派、2 路发射，服务 ALU0 和 ALU1；LSU IQ 为 32 项，支持 4 路分派、1 路发射；MD/BRU IQ 为 32 项，为乘法、除法和分支执行提供候选；Fast IQ 为 32 项，支持 4 路分派、2 路发射；Privilege IQ 为 16 项，只在指令位于 ROB 队首并满足精确执行条件时发射。分布式结构把候选比较限制在相关资源内部，减少全局选择网络的扇入。

发射项保存源物理寄存器标签、就绪位、操作信息和 ROB 编号。7 路完成总线向所有队列广播目的标签，匹配成功后更新对应源操作数的就绪状态；Store 数据也可通过旁路提前补入 SQ。由于完成广播和选择可能在相邻周期形成关键链路，队列内部以已登记的就绪状态为主，并结合必要的同拍旁路缩短依赖指令间隔。

5.2 执行资源与完成总线

普通结果/完成路径共 7 路：ALU0、LSU、MUL、DIV、Fast0、Fast1 以及 ALU1 共享路径。Privilege 操作复用 ALU1 的结果/完成端口，不构成第 8 路普通端口。乘法器采用固定 3 拍延迟，除法器采用固定 32 拍延迟；较长延迟操作在发射选择时受独立忙闲状态约束，不阻塞其他类型指令继续执行。

每一路完成信息至少携带 ROB 编号、目的物理寄存器标签、写回数据以及异常或控制流结果。ROB 按编号置完成，PRF 按 pdest 写入数据，所有发射队列按相同标签完成唤醒。完成次序可以与程序顺序不同，但提交始终由 ROB 队首决定。

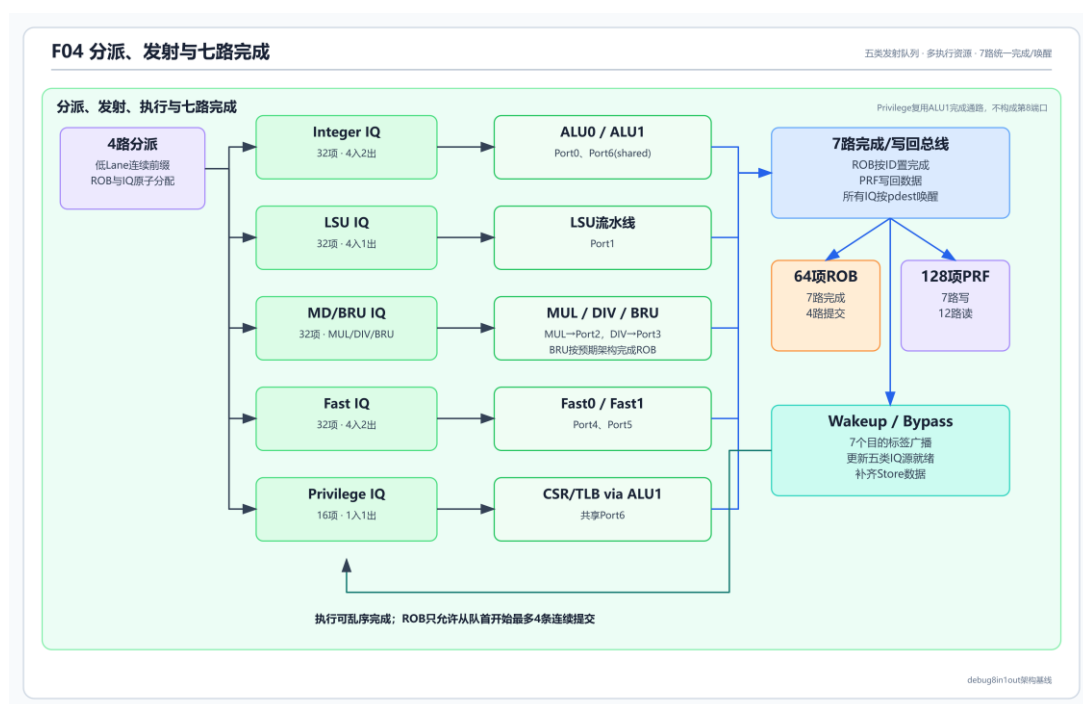


图 5-1 分派、发射、执行与七路完成关系

5.3 BRU 预期架构

本报告按预期架构描述 BRU。BRU 完成对应 ROB 项，并比较实际方向、实际目标与前端保存的预测快照；不一致时产生 `predict_flush`，给出正确目标 PC 和分支检查点编号。JIRL 除形成跳转目标外，还把 PC+4 写入目的物理寄存器；条件分支只产生控制流结果，不写物理寄存器。这样，控制流完成、链接寄存器写回和选择性恢复能够在同一套 ROB/PRF 协议中闭合。

6 ROB、四路提交与精确恢复

6.1 六十四项 ROB

ROB 包含 64 项，按照程序顺序循环分配。每项保存 PC、指令属性、新旧目的物理寄存器、完成状态、异常状态和恢复所需信息。执行单元可以乱序返回结果，但只能按 ROB 编号更新对应项，不直接改变提交指针。ROB 同时接收 7 路完成，并在队首检查最多 4 项是否构成可提交的低编号连续前缀。

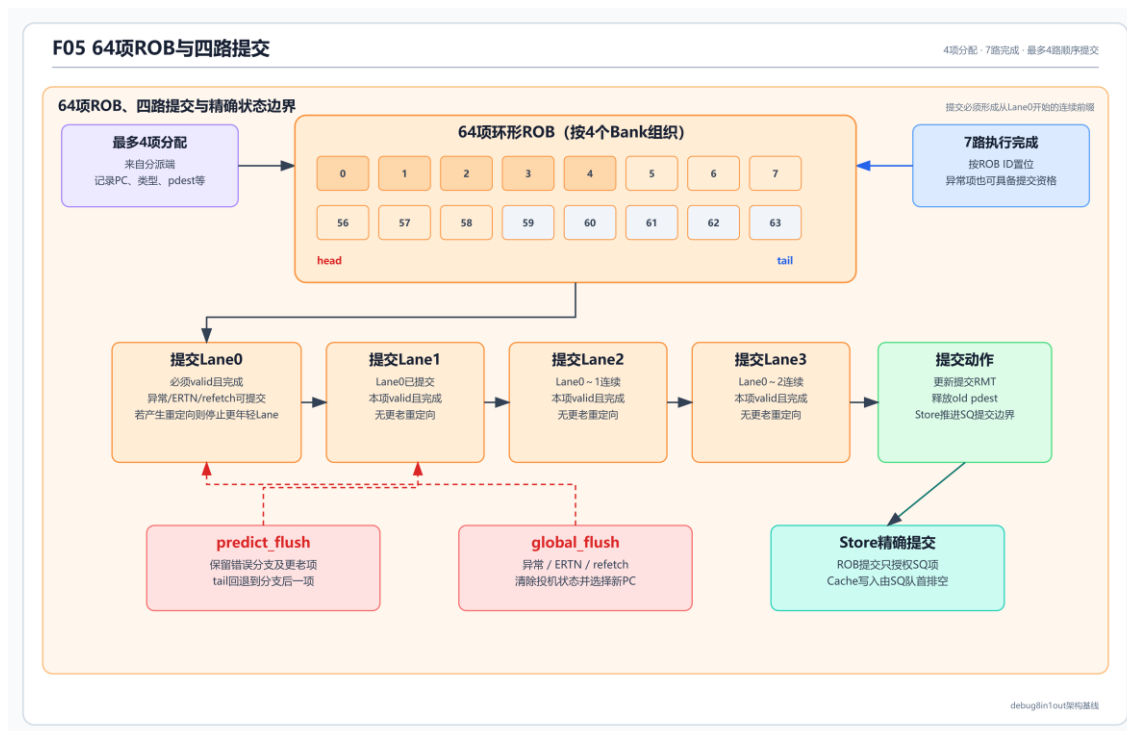


图 6-1 ROB 按序提交与精确状态恢复

6.2 四路提交

提交逻辑从 head 开始依次检查最多 4 条指令。只有第一条完成且无阻塞时才能考虑第二条，后续 Lane 同理；一旦遇到未完成、Store 尚不满足提交条件或需要恢复的指令，本拍更年轻指令均不能提交。普通寄存器写指令提交时更新提交 RMT 并释放 old pdest，Store 提交时把对应 SQ 项标记为 committed，分支提交时释放不再需要的检查点。

重定向或精确状态操作可以作为本拍最后一条提交指令，但会禁止它之后的年轻指令同拍提交。该规则避免恢复 PC、CSR 状态和提交 RMT 跨越体系结构边界，同时不必把触发恢复的指令本身重新执行。

6.3 两级恢复控制

局部 stall 只保持相关流水寄存器和上游状态，不改变程序路径。predict_flush 用于分支方向或目标预测错误：恢复对应分支的 RMT 检查点，清除该分支之后的年轻指令，并从 BRU 给出的正确 PC 重新取指。global_flush 用于精确异常、ERTN 和 refetch：恢复提交 RMT，清除全部年轻投机状态，并从异常入口、返回地址或重取地址开始执行。外部低有效 reset 则初始化有效位、指针、计数器和控制状态。



图 6-2 阻塞、预测恢复、精确恢复与复位控制

恢复信号由 ROB、前端、重命名、各发射队列和 LSU 共同消费。设计重点不是单纯“清空”，而是保留已经提交、仍应对体系结构生效的部分。例如 global_flush 不能丢弃已经提交但尚未从 DCache 端口排出的 Store；predict_flush 则必须只删除分支之后的年轻 LQ/SQ 项。通过统一 ROB 年龄比较和提交标记，各模块可以对同一恢复边界给出一致判断。

7 访存流水线与内嵌 LQ/SQ

7.1 LSU 流水组织

LQ 和 SQ 均集成在 LSU 访存流水线内部，不作为 LSU 外部的独立并行模块。指令从 LSU IQ 发射后，依次经过源操作数读取、AGU 地址生成与 MMU 转换、MEM1 访问准备、内部 LQ/SQ 处

理、MEM2 仲裁、DCache 访问/数据合并和写回。该组织使地址翻译、异常检查、内存相关判断和 Cache 访问共享统一时序边界。

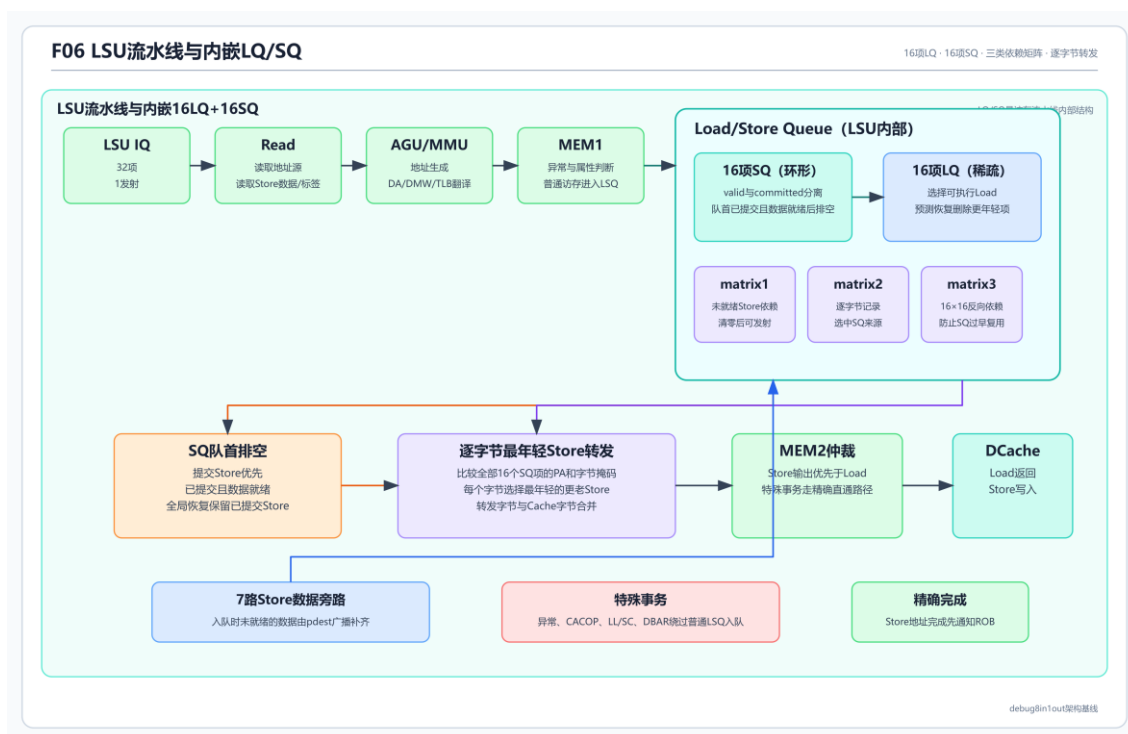


图 7-1 LSU 流水线以及内部 16 项 LQ 与 16 项 SQ

7.2 十六项 SQ

SQ 采用 16 项循环结构，分别维护 valid 和 committed 状态。Store 在分派/执行过程中占用 SQ 项，地址与字节写掩码由 AGU 产生，Store 数据可以从 PRF 读取，也可以由 7 路结果旁路补齐。只有队首 Store 同时满足 valid、committed、地址有效和数据就绪时，才允许向 MEM2/DCache 排出；因此投机 Store 不会提前修改存储系统。

predict_flush 删除错误分支之后的年轻 Store。global_flush 清除未提交 Store，但保留已经提交、尚待排出的 Store，确保精确异常不会撤销已经跨过提交边界的内存副作用。SQ 槽位只有在 Store 真正排出并解除相关依赖后才能复用。

7.3 十六项 LQ 与逐字节前递

LQ 为 16 项稀疏结构，用于登记在途 Load 及其相关状态。Load 在地址就绪后与全部 16 项 SQ 并行比较，先依据 ROB 年龄筛选更老 Store，再按地址与字节写掩码判断覆盖关系。对 Load 所需的每个字节，前递逻辑选择年龄最近且更老的匹配 Store；未被 Store 覆盖的字节来自 DCache，最终在写回前按字节合并。

若匹配到的更老 Store 尚未准备好数据，Load 必须等待而不能读取可能过期的 Cache 值。为防止 SQ 槽位循环复用后把旧依赖误认为新 Store，LSU 维护 16×16 反向依赖关系：尚有 Load 引用某个 SQ 槽位时，该槽位不能直接作为无关的新 Store 身份复用。地址冲突、数据未就绪、逐字节来源和槽位生命周期共同构成内存顺序检查。

7.4 特殊访存操作

普通 Load/Store 经过 LQ/SQ 完成投机管理；精确异常相关操作、CACOP、LL/SC 和 DBAR 等特殊指令则绕开普通 LSQ 快速路径，并在 ROB 队首或满足全局顺序条件后执行。MMU 异常在形成对 Cache 有副作用的请求前被截获，异常信息随 ROB 项提交，保证坏地址、权限错误和页无效等情况均以精确方式呈现。

8 Cache 层次与外部存储访问

8.1 一级 Cache

一级指令 Cache 和数据 Cache 均为 2 路组相联、128 组、32B 行，每个容量为 8KiB。32B 行与 256bit 内部传输宽度匹配，可一次提供 8 条指令或一整行数据片段。I-Cache 面向连续取指和预测重定向，D-Cache 面向字节、半字和字访问，并配合 LSU 完成写掩码、未对齐检查和 Load 数据选取。

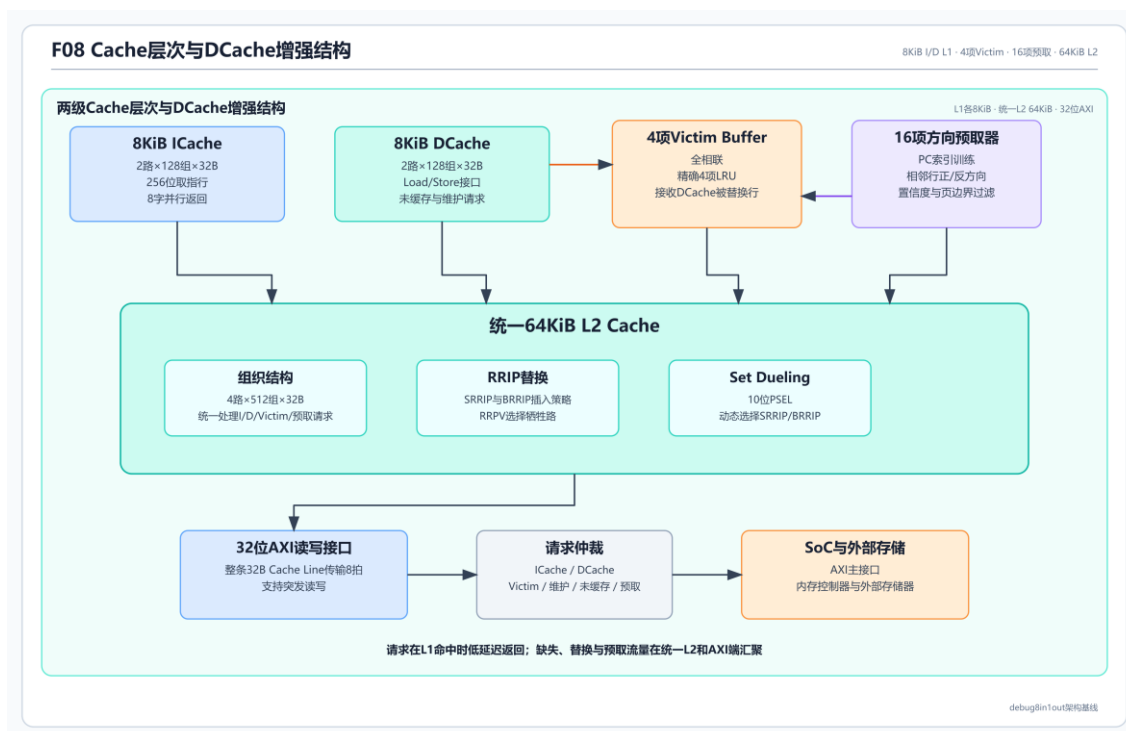


图 8-1 两级 Cache、Victim Buffer、预取与 AXI 访问层次

8.2 Victim Buffer 与预取

D-Cache 配置 4 项全相联 Victim Buffer，采用精确 LRU 管理被替换行。当主 Cache 缺失但 Victim Buffer 命中时，可避免立即访问 L2 或外部存储，并把近期冲突行重新带回一级结构。预取器包含 16 项以 PC 索引的相邻行方向状态，根据同一访存指令的连续访问趋势选择下一行或前一行进行预取。预取请求只使用空闲带宽，不能阻塞需求访问，也不能改变异常与一致性语义。

8.3 二级 Cache 与 AXI

L2 Cache 为 4 路组相联、512 组、32B 行，总容量 64KiB。替换策略采用 RRIP，并通过组决斗和 10bit PSEL 在候选插入策略之间进行选择，以兼顾扫描型访问与具有重用价值的数据。L1 请求、回写和预取在进入 L2 前进行仲裁，未命中请求再转换为外部 AXI 事务。

外部 AXI 数据宽度为 32bit，因此一个 32B Cache 行需要 8 个 beat 完成传输。读填充过程中维护 beat 计数和行内位置，全部数据收齐后再更新有效位与标签；写回同样按 8 拍发送。事务 ID、地址和响应状态在一次突发期间保持一致，错误响应转换为可由后端精确处理的访存异常。

9 MMU、TLB、CSR 与异常中断

9.1 地址转换顺序

MMU 按照直接地址模式、直接映射窗口和 TLB 转换的顺序选择物理地址。DA 模式直接使用虚拟地址形成物理访问；DMW0/DMW1 在权限和地址段匹配时完成窗口映射；其余地址进入 TLB 查找。转换结果除物理地址外，还携带存储属性和权限信息，分别供 I-Cache 与 D-Cache 决定访问方式。

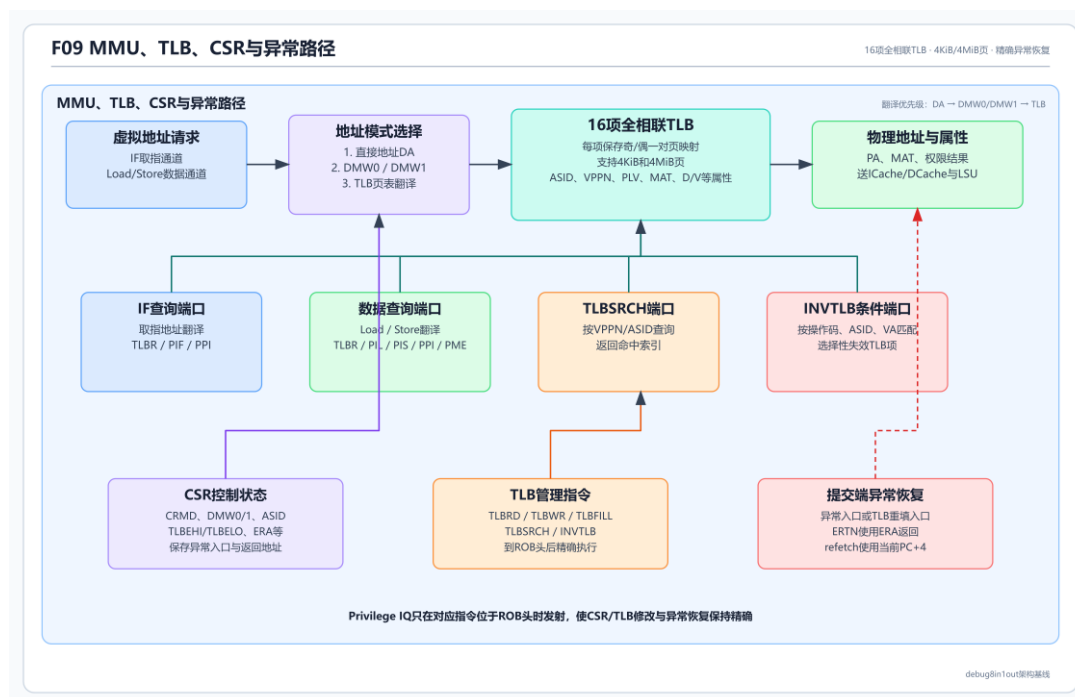


图 9-1 MMU、TLB、CSR 及精确异常恢复关系

9.2 十六项全相联 TLB

TLB 包含 16 项全相联表项，支持 4KiB 和 4MiB 页。取指与数据访问拥有独立查询角色，TLBSRCH 用于软件查找，INVTLB 按操作类型使匹配项失效。TLBRD 把选中表项读入 CSR，TLBWR 按指定索引写入，TLBFILL 按替换索引填充。页大小、虚拟页号、地址空间标识、全局位、物理页号、有效位、脏位和权限共同参与命中与异常判断。

9.3 异常判定

取指侧可能产生 TLB 重填、页无效和特权级不匹配异常；数据侧在此基础上区分 Load 页无效、Store 页无效和页修改异常。异常在访问产生副作用前被识别，并与 PC、坏地址和异常码一起写入 ROB。多个在途异常仍按照程序顺序处理，只有最老异常到达 ROB 队首时才触发 global_flush 和 CSR 更新。

9.4 CSR、ERTN 与 refetch

CSR 记录异常入口、返回地址、当前/先前特权和中断状态。精确异常提交时保存出错 PC 并跳转到异常入口；ERTN 恢复异常前的特权与中断状态，并使用 CSR 中的返回地址重新取指。某些 CSR 或 TLB 操作会改变后续指令对地址和权限的解释，因此在其精确提交后产生 refetch，清除年轻指令并从规定地址重新建立执行环境。

10 总结

本设计形成了从 8 字取指、4 路译码/重命名/分派、分布式动态调度、7 路普通完成到 64 项 ROB 四路提交的完整乱序处理器主链路。32 份分支 RMT 检查点支持预测错误的选择性恢复，提交 RMT 支持异常、ERTN 和 refetch 的精确恢复；LSU 内部的 16 项 LQ 与 16 项 SQ 以逐字节前递和反向依赖矩阵维护访存顺序；两级 Cache、Victim Buffer、方向预取、16 项 TLB 和 32bit AXI 接口构成完整存储系统。

从微架构配合关系看，前端宽度、乱序窗口和完成带宽已经形成闭环。8 字取指为 4 路译码提供带宽余量，128 项 FIFO 吸收预测路径和 Cache 返回的短时波动；64 项 ROB 与五类分布式发射队列共同扩大可见指令窗口，7 路完成广播则把执行结果及时传回 PRF、ROB 和等待队列。各结构的容量与端口不是简单叠加，而是围绕持续提供可提交指令这一目标协同配置。

恢复机制是该设计保持正确性的核心。分支错误只回退到对应 RMT 检查点，减少无关状态重建；异常、ERTN 和 refetch 回到提交 RMT，确保体系结构寄存器、CSR 和取指地址来自同一个精确边界。ROB 年龄、低编号连续提交和统一清除边界贯穿前端、发射队列及 LSU，避免不同模块各自解释“年轻指令”而造成状态分歧。

访存部分采用集成式 LSU 组织，把地址生成、地址翻译、内部 LQ/SQ、DCache 访问和 Load 合并放在同一条流水线上。16 项 SQ 以提交状态限制 Store 副作用，16 项 LQ 通过全 SQ 比较获得逐字节最近老 Store 数据，反向依赖矩阵进一步保护循环槽位身份。该结构使 Load 可以在满足顺序条件时提前获得数据，同时仍保持异常和 Store 对外可见次序精确。

存储层次的设计重点是降低一级缺失的平均代价。8KiB 一级指令/数据 Cache 提供低延迟访问，Victim Buffer 缓解冲突缺失，方向预取器利用 PC 相关的相邻行趋势，64KiB 二级 Cache 再以 RRIP 和组决斗适应不同重用模式。所有优化请求最终通过统一仲裁汇入 32bit AXI 接口，需求访问优先级和异常语义不会被预取改变。

当前成绩基线表明功能测试 58/58 通过，系统能够启动 Linux 并运行基本命令，20 项性能程序均获得有效计数。后续优化应在不改变精确状态边界的前提下，围绕分支预测有效命中率、关键完

成—唤醒路径、Load 等待判定以及 L2 未命中代价展开，并继续以系统计数比、IPC 比和实现频率的组合指标评价优化收益。

后续验证将继续采用“模块不变量—流水级接口—系统程序”三级方法。模块级关注指针、计数和年龄比较不变量；接口级检查有效/允许握手、恢复优先级和多端口同拍行为；系统级以功能、启动和性能计数交叉验证。性能改动只有在功能测试、精确恢复和访存顺序均保持稳定后才进入成绩对比，以避免用局部吞吐换取难以复现的体系结构错误。

参考文献

- [1]龙芯中科.龙芯架构 32 位精简版参考手册 V1.03[EB/OL].[2025-08-18].
- [2]姚永斌.超标量处理器设计[M].北京:清华大学出版社,2014.
- [3]汪文祥,邢金璋.CPU 设计实战[M].北京:机械工业出版社,2021.
- [4]胡伟武,汪文祥,等.现代 CPU 设计[M].北京:机械工业出版社,2026.